

manabot: A Research Platform and Open Challenge Set for *Magic: The Gathering*

Jack Heart
Loopflow Studio

July 9, 2026

Working draft — a standing snapshot of the platform and the open frontier.

Abstract

Magic: The Gathering combines an expanding card pool, structured hidden information, state-dependent actions, long decision horizons, and a rules system capable of universal computation. Existing work has studied search, deck construction, card representations, and, recently, causal reinforcement learning in restricted versions of the game. This work focuses on a complementary combination: a fast, rules-explicit environment with native search primitives and cost-calibrated baselines.

This paper presents **manabot**. We contribute (i) **managym**, a deterministic, seedable, forkable Rust engine ($\approx 15\text{k}$ lines, 162 rules-cited integration tests, a 55-card pool) that runs 1.8×10^5 steps/second on an Apple M4 Max and exposes determinization and state injection as native primitives; (ii) a **measurement protocol** based on seat-balanced evaluation, Wilson intervals, pre-registered predictions, and standardized compute accounting; (iii) a **reference suite** spanning flat determinized search, PPO under three reward regimes, behavior cloning from search, and one measured round of expert iteration; (iv) a **control-competency suite** — five injected scenarios with known-correct lines — that no current agent passes; and (v) **five open challenge areas** with measurable entry criteria.

On an interactive mirror, SEARCH-16 requires no training and runs in 7 ms per decision. It wins 91.7% against a uniform-random opponent and defeated each of the three early PPO policies tested head-to-head. A student distilled from SEARCH-256 reaches 87.0% against random at ladder strength $N \approx 7$; one crank of expert iteration on top of it *degraded* the student, and the search pilot itself scores at most 0.39 on the five competency scenarios — uniform random beats it outright on three. These are baseline results, not a claim to strong *Magic* play: there is no human evaluation, adaptive opponent population, or broad card pool. They expose a large gap on the common random-opponent axis and define the tests a solution must pass.

1 Introduction

Sequential decision-making research depends on two kinds of infrastructure: a simulator fast enough to support iteration, and a protocol that makes its numbers interpretable. Until both exist, algorithmic comparisons mostly measure implementation and evaluation choices. The environment is not merely scaffolding around the science; for a period, it is part of the science.

Magic: The Gathering is a natural target. It is imperfect-information and long-horizon, and its full rules admit universal computation [Churchill et al., 2021]. In this benchmark’s open-decklist setting, public decklists constrain hidden cards to known multisets: the support is combinatorial but explicit, and exact samples from the turn-one prior are cheap. Later in a game, actions make

the posterior policy-dependent. That combination gives belief modeling both a clean prior and a meaningful inference problem.

Prior work includes Monte Carlo search for card selection [Ward and Cowling, 2009], ensemble determinization for game play [Cowling et al., 2012b], and generalized representations for unseen cards in deck construction [Bertram et al., 2024]. A contemporaneous preprint introduces MTG-Causal-RL, a 56-card, five-archetype Gymnasium benchmark with a fixed masked action space and an explicit causal interface [da Costa Cunha et al., 2026]. Manabot is complementary: it emphasizes a rules-cited Rust engine, state-dependent object-referential actions, native determinization and search, and a baseline ledger that includes seat effects, inference cost, and failed predictions.

The remaining entry cost is still high. Restricted environments necessarily choose a rules slice; evaluation can be dominated by seat and opponent effects; and a learned policy can look strong against random play while remaining weaker than a small search budget. We encountered all three problems in that order.

This paper is an attempt to pay that cost once, publicly, so that the next person does not have to. Sections 3–4 describe the platform and measurement protocol. Section 5 gives the reference baselines — what currently works, what it costs, and where measured agents sit on a common strength ladder. Section 6 adds the behavioral lens: a five-scenario control-competency suite, with known-correct lines, that every current agent fails. Section 7 turns the exposed gaps into five entry criteria; Section 8 bounds their interpretation; and Section 9 records the calibration findings that changed the project.

What we are claiming. The environment is fast, deterministic, instrumented, and covered by CR-cited tests within a stated subset; the reported baselines follow one explicit protocol; and the resulting gaps motivate five concrete research targets. Tests do not prove rules correctness, and the protocol has not been validated beyond the reported decks and opponents. We are explicitly *not* claiming to have trained a strong agent. That is the invitation.

2 Why *Magic* Is Hard, Precisely

Difficulty claims about games are cheap. Here is what is actually load-bearing about this one, in the form of the constraints each property places on a solution.

The action set is variable, state-dependent, and object-referential. At each decision the legal set is a list — cast this, block that with this, target the other — of size 1 to 32 in our subset, and index i carries no stable meaning across states. A fixed maximum-width masked head is possible, and is common in related environments, but it discards the semantics shared by actions that refer to the same objects. Manabot instead scores each legal action from its type and referenced objects. The mean is 3.08 valid actions per surfaced decision on the original two mirrors; the tail and the changing referents create the modeling problem, and the tail grows with the pool — the constructed decks of §3.5 exceed the original 20-action encoder capacity at real priority windows.

The hidden state has explicit support. With public decklists, the unknown state is the opponent’s hand and the order of both libraries. A uniform sample from the opponent’s unseen pool is correct before their behavior conveys information; it becomes misspecified as observed actions reveal their policy and likely holdings. Determinization has long been used for *Magic* search [Cowling et al., 2012b]. The open opportunity here is to condition those samples on public action history and measure whether the improved posterior produces stronger play. This is Challenge 2.

The learning horizon exceeds the credit window. In random-versus-random play, games run 33–41 turns and surface about 58–97 decisions to one player, depending on the deck. PPO with $\gamma = 0.99$ and $\lambda = 0.95$ has an effective GAE horizon of $1/(1 - \gamma\lambda) \approx 16.8$ agent steps, so the measured trajectories span roughly 3.5–5.8 such horizons. Simple per-event shaping is harmful on the interactive deck (§9); potential-based shaping does not establish an improvement over terminal-only reward at the tested budget. Long-horizon learning therefore remains open, but the current experiments do not identify its solution.

The card pool is expanding and semantic. We implement 55 cards; the full game contains tens of thousands and continues to add mechanics. An identifier-only policy cannot transfer to an unseen card. Useful representations must connect rules text or structured card semantics to game behavior, an issue already studied for deck construction [Bertram et al., 2024] but not for play in this platform. This is Challenge 4.

The variance is enormous and the confounds are worse. Between two *uniformly random* players on a creature deck, going first wins 94.1% [92.5%, 95.4%] of games. Any evaluation that does not balance seats is measuring the seat. We discovered this after it had invalidated our own founding result (§9).

3 The Platform

3.1 managym: the engine

A two-player subset of the *Comprehensive Rules* in Rust (14,923 lines), with PyO3 bindings. Table 1 states the implemented rule families and known omissions. The 162 CR-cited integration tests exercise the listed behavior; they are evidence about the subset, not a proof of complete rules correctness.

CR section	Status	Notes / known gap
103 Game start	tested	Opening hand of seven; no mulligans
106 Mana	tested	Deterministic auto-tap; no floating-mana choice
117 Priority	tested	APNAP pass loop
305 Lands	tested	One per turn; no additional-land effects
405 Stack	tested	LIFO resolution, priority after resolution, counterspells
601 Casting (subset)	tested	Timing/cost legality; kicker, cost reduction, multi-target; mid-resolution choices (scurry, look-and-select, pay-or-not, modal, learn)
602 Activated abilities (subset)	tested	Stack use, costs, duration, source removal; waterbend tap-to-pay
603 Triggers	tested	Data-driven substrate; intervening-if; delayed triggers; §3.4
605 Mana abilities (subset)	tested	Triggered mana abilities (605.1b); until-end-of-combat pools
613 Continuous effects	tested	Simplified additive slice: base/CDA → statics → until-EOT → counters; no timestamps or dependency
702 Keywords (subset)	tested	Thirteen combat and timing keywords, ward, until-EOT grants
508/509 Attack, block	tested	Per-creature declaration; multiple blockers; no damage-order choice
510 Combat damage	tested	First/double strike, trample, deathtouch, menace
514, 704.5 Cleanup, SBAs	tested	Damage cleanup; life, draw, and lethal-damage losses; zero-toughness deaths
614 Replacement/prevention	absent	Designed, unimplemented. Challenge 5
Multiplayer, planeswalkers, battles	absent	<code>Game::new</code> asserts two players

Table 1: Rules coverage. “Tested” means CR-cited integration tests exist in the rules-test directory; there are 162 in total. The table summarizes supported rule families rather than every implemented clause. The absences bound every claim in this paper.

The pool is 55 real cards plus two token definitions: five basic lands and 18 spells from Alpha (*Lightning Bolt*, *Counterspell*, *Ancestral Recall*, *Shivan Dragon*, ...), *Pyroclasm*, *Man-o’-War*, four Strixhaven cards, and a 26-card *Avatar: The Last Airbender* slice exercising tokens, +1/+1 counters, flash, ETB triggers with subject predicates, and the Milestone-1 mechanics of §3.5. Every registered shell is checked against a committed Scryfall snapshot in CI (§9 explains why that tripwire exists). Two 60-card mirror decklists are standard for calibration:

- STANDARD — 12 Mountain, 12 Forest, 18 Llanowar Elves, 18 Gray Ogre. Zero interaction: a pure race.
- INTERACTIVE — 12 Island, 12 Mountain, 20 creatures, 16 interaction spells (6 *Lightning Bolt*, 4 *Counterspell*, 3 *Ancestral Recall*, 3 *Pyroclasm*). Instant makes timing matter.

Having two decks with opposite strategic character is not decoration. We show in §9 that agents trained with terminal reward alone develop different aggregate behavior on them — casting nearly

always on one, holding much more often on the other. This paired profile is the sharpest test currently available in the platform for distinguishing deck-sensitive behavior from a fixed habit.

3.2 Determinization as a primitive

This is the design decision that makes the platform useful for imperfect-information research. All randomness flows through one `ChaCha8Rng`, so `(decklists, seed)` determines a game exactly. `Env::fork()` deep-clones an independent game. And `determinize(seed, perspective)` samples a world from the current uniform hidden-card model: it pools the opponent’s hand with their library, shuffles, re-deals to preserve the observed hand size, reshuffles the perspective player’s library, and preserves every public zone and the current action space.

On top of this the engine ships `flat_mc_scores`, which for each legal action samples W determinized worlds, applies the action to a clone, and scores by R uniform playouts to a terminal or step cap, sharing worlds across actions as common random numbers. **Search is an engine primitive, not a Python loop.** The full search sweep in §5 used 4.5 CPU-core-hours. The sampling boundary is localized inside `determinize`, which gives Challenge 2 a clear integration point even though a learned posterior still requires its own history encoder and inference path.

Listing 1: A minimal imperfect-information search loop. Signatures are current as of this draft.

```
import managym

deck = {"Island": 12, "Mountain": 12, "Gray Ogre": 6, "Wind Drake": 6,
        "Man-o'-War": 4, "Raging Goblin": 4, "Lightning Bolt": 6,
        "Counterspell": 4, "Ancestral Recall": 3, "Pyroclasm": 3}
hero    = managym.PlayerConfig("hero", deck)
villain = managym.PlayerConfig("villain", deck)

env = managym.Env(seed=0, skip_trivial=True)
obs, info = env.reset([hero, villain])

# Drive a sampled world yourself without changing env:
scratch = env.clone_env()
scratch.determinize(seed=7)
scratch.random_playout(seed=7)

while not obs.game_over:
    # One line of imperfect-information search: 16 worlds x 4 rollouts per
    # legal action, scored by uniform playouts to a true terminal.
    scores, simulations, cap_hits = env.flat_mc_scores(
        worlds=16, rollouts=4, seed=1, max_steps=2000)
    action = max(range(len(scores)), key=scores.__getitem__)
    obs, reward, terminated, truncated, info = env.step(action)
```

3.3 Throughput, and where the time actually goes

Measured on an Apple M4 Max. Environment stepping alone: 90,898 steps/s at one environment, $183,350 \pm 660$ at sixteen, saturating near 1.9×10^5 . Add policy inference and the original harness collapsed to 2,035 steps/s; with the full PPO update, 2,472–2,856. At that point inference cost roughly $72\times$ one environment step, and we wrote that “the environment is not the bottleneck.”

That statement was true but incomplete: most of the inference tax was self-inflicted. Three layers of fix — removing a debug tax from the agent’s hot path (eagerly formatted logging forcing device syncs, per-call mask allocations, a per-sample Python loop), a batched driver that runs one

masked forward for many concurrent games over the zero-copy buffers the Rust envs write into, and moving that batched forward to the M4’s GPU — lifted net-in-the-loop throughput from 2,035 to **24,474** obs/sec (12.0×; 6.4× against a same-day re-measurement of the baseline under identical machine load). The layers are not independent: MPS at batch 16 is useless and only becomes $\approx 6\times$ CPU at batch 256 and above, so batching is what makes the GPU usable at all; CPU-only the model is genuinely compute-bound at $\approx 7.3\text{k}$ obs/sec. Two conditional facts matter for anyone reproducing this: **MPS does not multiplex across processes** — four workers sharing the GPU deliver roughly one worker’s throughput — so in-the-loop data generation must batch across games within one process, and the same 400-game evaluation that took ~ 10 minutes of process-pool time now runs in 9 seconds.

Torch’s share of the loop fell from 97.2% to 74.8%; the environment is visible in the profile again (25.2%) for the first time since the engine work, and the next lever is observation-transfer consolidation, not further network optimization. All numbers trace to `reports/exp-07-expert-iteration.md`.

3.4 The trigger substrate

Triggered abilities are data, not code: `Ability::Triggered{condition, effects}`, where `TriggerCondition` ranges over ETB / dies / attacks / becomes-tapped / tapped-for-mana / upkeep / “you draw your n th card this turn”, each parameterized by a `TriggerSubject` (`This`, `AnotherYouControl(pred)`, `AnyYouControl(pred)`) over a structural `CardPredicate`. Effects compose, including a recursive `OnNthResolutionThisTurn` gate. Triggers are collected from game events, ordered APNAP, dropped when no legal target exists (CR 603.3d), and placed on the stack.

This substrate is the bet on which card-pool growth rests. Milestone 1 (§3.5) was its first real test: the 26-card cube slice landed on this substrate without changing its shape, and the audit that followed found the errors in hand-transcribed card *data*, not in the trigger machinery. Whether it continues to scale is Challenge 5.

3.5 Milestone 1: two constructed decks from a real cube

The platform’s first pool expansion was deliberately shaped like the real task: take two 40-card decks from an actual cube design — UR LESSONS (counterspells, card selection, kicker) and GW ALLIES (tokens, +1/+1 counters, an anthem, earthbend) — and implement everything they need. That forced four capability families into the engine:

- **Mid-resolution choices.** A resolving spell or ability can park itself and surface a decision — scry, look-and-select, pay-or-not (kicker, ward), modal, learn — as a first-class action space; nested decisions compose. Search stays coherent through these: cards revealed by a pending decision are pinned during determinization.
- **Continuous effects.** A simplified CR 613 slice (base and characteristic-defining P/T, then static effects, then until-end-of-turn deltas, then counters; every Milestone-1 effect is additive, so timestamps are not modeled), plus earthbend’s land animation with a delayed return trigger and exile-until-the-source-leaves linkage.
- **Cost mechanics.** Affinity-style reduction, waterbend (tap permanents to help pay), and triggered mana abilities that fire mid-payment.

- **Scenario injection.** `Env.scenario_*` methods (set life, force cards in hand, force battlefield permanents, refresh) construct exact mid-game positions, bypassing the rules engine. This exists for measurement, and §6 is built on it.

Both decks are playable in the browser UI against any agent, with MTGO-style priority stops and auto-pass. The first cross-deck matchup table (2,000 seat-balanced games, zero engine errors) is a calibration result discussed in §9: GW Allies wins 65.5% under random-vs-random and $\approx 77\%$ in search mirrors — and §6 shows why that last number says more about the pilot than the decks.

3.6 manabot: the learning side

A Gymnasium-compatible vectorized wrapper [Towers et al., 2024] and an object-centric observation encoder used by PPO and the distilled policy. The flat-MC player instead reads raw engine state; the distillation harness connects the two by recording encoded observations paired with search actions.

Observations are designed to respect hidden information — the opponent’s hand and both libraries are never encoded — and are object-centric: per-player [28], per-card [60×38], per-permanent [40×24] (including effective P/T and effective keywords, so anthems, counters, and until-end-of-turn grants are agent-visible), actions [32×15], events [32×7], plus `action_focus` [32×2], which maps each action to the (up to two) objects it refers to. Validity masks accompany every padded tensor. These shapes have grown twice with the rules substrate; each growth invalidates earlier checkpoints, a bookkeeping fact that matters when comparing agents across sections (§5.1).

The policy embeds each object class into $d = 64$, optionally mixes them with one transformer block [Vaswani et al., 2017] carrying a learned *perspective* vector whose sign flips by ownership, then scores each action by concatenating its embedding with those of its focus objects and passing the result through a shared head. One logit per action slot; invalid slots masked to -10^8 . It is deliberately *not* a pointer network [Vinyals et al., 2015]. 100,354 parameters (50,306 without attention). We regard this architecture as a placeholder, not a contribution.

PPO is CleanRL-style [Huang et al., 2022, Schulman et al., 2017] with GAE [Schulman et al., 2016]: $\gamma = 0.99$, $\lambda = 0.95$, lr 2.5×10^{-4} annealed, 16 envs $\times$ 128 steps, 4 minibatches, 4 epochs, clip 0.1. Three reward regimes are selectable: terminal-only; per-event (“pay-per-event”: +0.03 land, +0.06 creature, +0.01 per point of opponent life); and potential-based [Ng et al., 1999], $\gamma\Phi(s') - \Phi(s)$ with matched weights. The results in §9 support terminal-only reward as the current default.

Search and distillation. `FlatMCPlayer` exposes a single strength dial $N =$ simulations per legal action. Distillation is pure behavior cloning from SEARCH-64 self-play: record every decision as (observation, teacher’s arg max), minimize cross-entropy through the student’s masked logits, split train/validation *by game*. No DAGger [Ross et al., 2011], no soft targets, no value head — all three are open.

4 The Measurement Protocol

We state this before the results because in this domain it *is* a result, and because an entrant who ignores it will produce numbers that do not mean anything. We know, because we did.

Seat balancing is mandatory. The engine gives `PlayerId(0)` the first turn. Between two uniformly random players:

Deck	On the play	On the draw	Overall (balanced)
STANDARD	94.1% [92.5, 95.4]	5.9%	50.3% [47.2, 53.4]
INTERACTIVE	23.1% [20.6, 25.8]	76.9%	48.9% [45.8, 52.0]

On the creature deck, moving first wins 94% of games between two uniform-random policies. On the interactive deck the seat effect *inverts*: the on-play policy wins only 23.1%. Either way, an unbalanced win rate is primarily a reading of the seat. All reported evaluation win rates alternate the hero between seats and include overall and per-seat rates with Wilson intervals [Wilson, 1927].

The gate. A policy *passes* if, over ≥ 400 seat-balanced games against a uniform random opponent on INTERACTIVE, its Wilson 95% lower bound exceeds 55% overall *and* its win rate exceeds 50% in each seat separately. The second clause exists to reject seat-parasitic policies, which we produced immediately upon looking for them.

Ladder strength. Win rate against random saturates and cannot rank strong agents. For a search-free policy, we report **estimated ladder strength**: the largest evaluated N for which it has a head-to-head point estimate of at least 50% against SEARCH- N . Every estimate is reported with its interval; a rung is *confirmed* only when the Wilson lower bound exceeds 50%. The search policies occupy their own reference rungs by construction. The search family improved monotonically in the evaluated comparisons, including a direct result in which SEARCH-256 beats SEARCH-16 76.7% [71.6, 81.1]. The ladder is training-free and N is interpretable as a simulation budget, but monotonicity outside the measured rungs is an assumption rather than a theorem.

The cost axis. Strength is meaningless unbudgeted. We book measured wall-clock time at an AWS `g5.xlarge` on-demand rate of \$1.006/hr. An early measurement priced 10^6 steps at \$0.44; the later vectorized harness ran roughly four times faster, so the ledger uses each experiment’s realized time rather than a universal per-step conversion. This is an accounting proxy, not an incurred cloud bill: all experiments ran on one laptop. Hardware, batching, and pricing changes limit comparisons outside this paper.

Pre-registration. Each experimental cycle opens with a written question, a prediction, and a cost cap, and closes with a report in **reports/** recording whether the prediction held. Of seven predictions registered by the first four cycles, three were refuted outright; the cycles since have refuted four more, including both halves of the expert-iteration prediction (§5.1), the claim that search would narrow a cross-deck gap, and the claim that a control deck’s win rate would stay flat in search strength (§6). We publish all of them.

5 Reference Baselines

Table 2 is the reference suite. Every row is 400 seat-balanced games against a uniform random opponent on INTERACTIVE, except the search rows (300 games) and the mirror row. These rows were measured before the Milestone-1 observation-shape changes; §5.1 adds the current-world frontier on top of them.

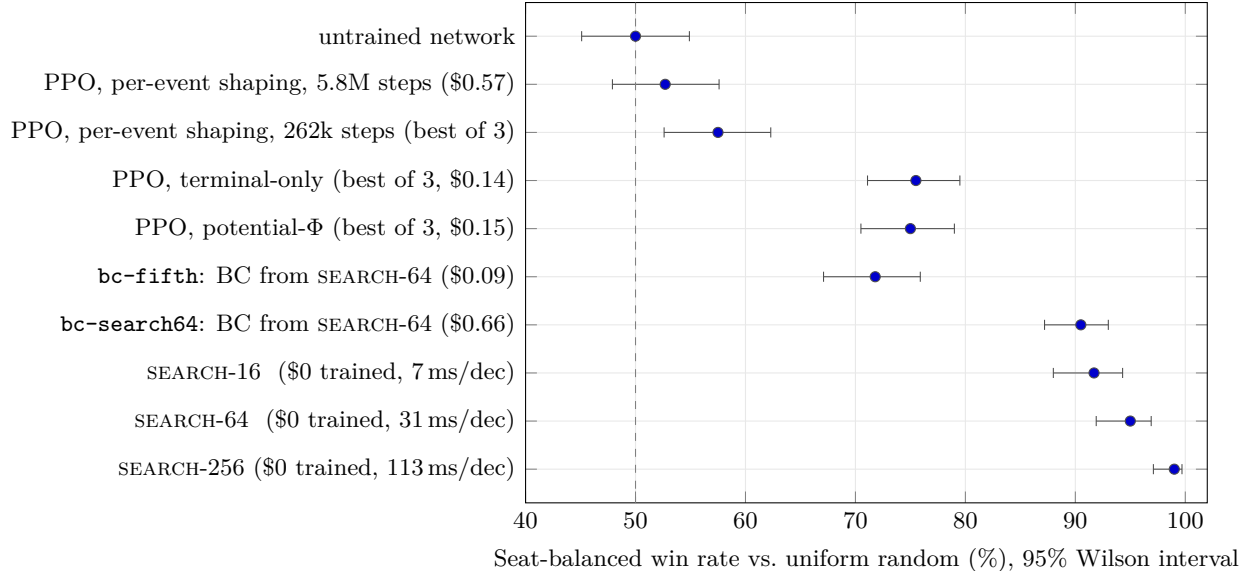


Figure 1: The current frontier against a common random opponent. Every baseline stronger than **bc-search64** on this axis is a search policy. The gap between the best learned-from-scratch point estimate (75.5%) and the cheapest search (91.7%) motivates Challenge 1; it is not a head-to-head result.

Agent	Train \$	ms/dec	Win % vs. random	Ladder strength
uniform random	0.00	~0	48.9 (mirror)	—
untrained network	0.00	1.0	≈ 50	unmeasured
PPO, per-event, 262k	1.30 [†]	1.0	57.5 [52.6, 62.3]	< 16
PPO, per-event, 5.84M	0.57	1.0	52.7 [47.9, 57.6]	unmeasured
PPO, potential- Φ	0.15	1.0	75.0 [70.5, 79.0]	unmeasured
PPO, terminal-only	0.14	1.0	75.5 [71.1, 79.5]	unmeasured
bc-fifth	0.09	0.98	71.8 [67.1, 75.9]	unmeasured
bc-search64	0.66	0.98	90.5 [87.2, 93.0]	N $\approx$ 8
SEARCH-16	0.00	7	91.7 [88.0, 94.3]	16 (by construction)
SEARCH-64	0.00	31	95.0 [91.9, 96.9]	64
SEARCH-256	0.00	113	99.0 [97.1, 99.7]	256

Table 2: Reference baselines. PPO rows show the best of three seeds; means for per-event and potential shaping are 52.4% and 69.5%. [†]Cumulative across the training program. Ladder values use the point-estimate definition in §4; “unmeasured” means the necessary head-to-head matchups have not been run.

Three facts organize everything that follows.

A small search budget dominates the early PPO baselines. The search experiment covered 13 matchups and 3,900 seat-balanced games (77.9M playouts, 4.5 CPU-core-hours, \$0 of training). Against the three per-event PPO checkpoints available at the time, SEARCH-16 wins 84.0–92.7% overall; across all nine search-versus-PPO matchups, no per-seat rate falls below 80.7%. This result does *not* place the later terminal-only or potential-shaped PPO policies, whose ladder matchups remain unmeasured. The searcher’s `cast_when_able` rate falls from 0.64 at $N = 16$ to 0.52 at

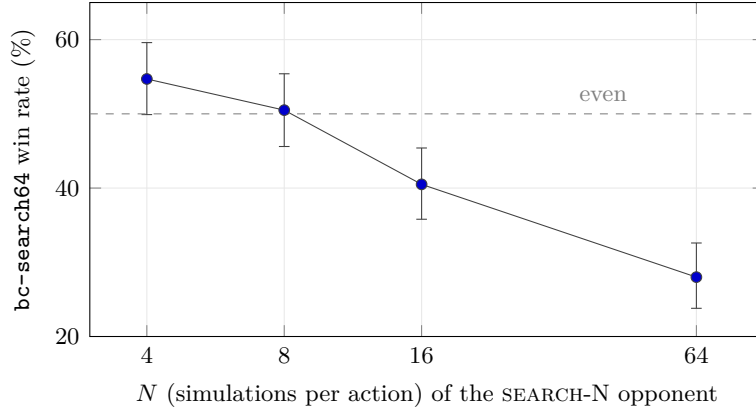


Figure 2: How to read estimated ladder strength. The distilled student’s point estimate is 50.5% at $N = 8$ and 40.5% at $N = 16$, so we report $N \approx 8$; its interval at $N = 8$ includes 50%, so the rung is not confirmed. The later PPO baselines have not been placed. Challenge 1 asks for a policy that clears $N = 16$ with a lower bound above 50%.

$N = 256$, showing that deeper search in this deck increasingly values waiting.

Distillation beats the cost-matched PPO recipe, with a confound. Behavior cloning on 73,443 decisions from 600 SEARCH-64 self-play games (\$0.66) reaches 90.5% against random and estimated ladder $N \approx 8$ at 0.98 ms per decision — a 32× inference speedup over its teacher, with performance near the rung that uses one eighth of the teacher’s simulation budget (Figure 2). A wall-clock-matched PPO run costs \$0.57, processes 5.84M timesteps, reaches 52.7% against random, and loses to the student 82.0% [77.9, 85.5] head-to-head. A cheaper clone costs \$0.09 and reaches 71.8%.

The matched PPO used the per-event shaping recipe that §9 later found to be harmful. The strongest terminal-only PPO point estimate is 75.5% at \$0.14, but it was not trained at matched cost or compared head-to-head, and the distillation model has twice as many parameters. The defensible conclusion is therefore narrower: search supervision produced the strongest learned policy in this suite, while the reported experiment does not isolate supervision, architecture, and reward design well enough to establish a general cost-efficiency multiplier.

The student inherits an aggregate behavior. `cast_when_able` is 0.58 for the SEARCH-64 teacher, 0.61 for its student, and 0.97 for the cost-matched per-event PPO. Terminal-only and potential-shaped PPO also learn to wait, with rates of 0.15–0.29 and 0.35–0.38. The close teacher–student match is evidence of behavioral transfer, but this proxy alone does not establish why either policy waits.

5.1 The second generation: distillation from search-256, and a failed crank of expert iteration

The results above predate the Milestone-1 observation-shape changes; every checkpoint behind them is dimensionally dead in the current engine. The first cycle run in the current world (`reports/exp-07-expert-iteration.md`) re-established the distillation baseline at a higher teacher budget and then attempted the obvious next step — closing the AlphaZero-style loop

[Anthony et al., 2017, Silver et al., 2018]. Both halves of that attempt were pre-registered, and both were refuted.

The round-0 student. Behavior cloning on 600 SEARCH-256 self-play games (80,568 decisions, hard argmax targets) yields `student_r0`: 87.0% [83.3, 89.9] against random over 400 seat-balanced games, at estimated ladder strength $N \approx 7$ (it splits with SEARCH-8, 44.5% [37.8, 51.4]), for roughly \$2.90 of data generation and training. A temperature sweep over soft targets $p \propto \exp(\text{score}/\tau)$ found strength *monotone in target sharpness*: hard argmax won (88.0% on the quick evaluation), $\tau = 0.02$ scored 84.0%, $\tau = 0.05$ scored 80.0%, and larger τ was clearly worse. At 256 simulations per action the argmax is already low-noise; softening mostly re-injects the teacher’s rollout noise floor.

Policy rollouts do not pay for themselves. The prediction: replacing random rollouts inside the searcher with the trained student would win at equal wall-clock, because rollouts would stop wasting interaction. At equal *simulations* the policy-rollout searcher shows a positive but non-significant edge (56.0% [46.2, 65.3]). But a policy rollout costs $\approx 6\times$ more per playout even with hybrid random tails, and at equal *wall-clock* — the honest comparison — the random-rollout searcher wins decisively: the policy-rollout side took **31.0%** [22.8, 40.6].

The crank degraded the student. One full round of expert iteration — the student becomes the rollout policy for a new teacher, whose labels train a new student — was predicted to beat `student_r0` head-to-head and place at ladder $N \geq 16$. Instead the round-1 student *lost to its predecessor* 74.2–25.8 (25.8% [21.7, 30.3]) and fell to ladder $N \approx 3$ –4. The diagnosis is label economics, the same fact as the wall-clock result seen from the other side: the affordable policy-rollout teacher could argmax only 8 noisy policy playouts per action, while round 0 had argmaxed 256 cheap random playouts — and **argmax-of-8-smart is a worse label source than argmax-of-256-dumb**. A 0.21 ms random playout buys more label truth per dollar than a ~ 34 ms policy playout. The refutation convicts this cycle’s affordable loop, not expert iteration in principle — but it is a calibration result for the field: *at this scale, the loop does not compound by default*. Anyone budgeting an expert-iteration project on a fast engine should price the label source, not the algorithm.

6 The Control-Competency Suite

Everything in Section 5 is a strength measurement: who beats whom, at what cost. This section is a different kind of instrument, and we believe it is the platform’s most useful recent contribution: a *behavioral* benchmark with known-correct answers, built after the win-rate lens visibly failed.

The failure was concrete. The first cross-deck matchup table (§3.5) put the control-shaped deck at $\approx 22\%$ against the aggressive deck in search mirrors — and search *widened* the gap relative to random-vs-random, the opposite of the pre-registered prediction that search would reclaim the control deck’s conditional value. Two hypotheses survive that table: the deck is structurally worse, or the pilot — flat determinized MC with uniformly random rollouts — cannot play control at all. Win rates cannot distinguish them. Scored decisions can.

The instrument. Five constructed mid-game positions, injected exactly via the engine’s scenario surface, each with a scripted opponent and a documented, information-sound correct line. Each agent plays each scenario 100 times; the score is the fraction of runs on the correct line. The

suite runs in about four minutes (`manabot/verify/competency.py`), and its predictions were pre-registered.

Scenario	Correct line	random	SEARCH-16	SEARCH-64	SEARCH-256
S1 counter-the-bomb	save the counter for the bomb	0.34	0.39	0.30	0.39
S2 hold-the-wipe	wait one turn, wipe four	0.23	0.00	0.00	0.00
S3 bolt-the-threat	hold removal for the threat	0.00	0.00	0.00	0.00
S4 race-vs-block	keep the blockers back	0.04	0.02	0.01	0.00
S5 hold-up-quench	hold mana, counter the threat	0.14	0.01	0.08	0.03

Table 3: Correct-line rate, 100 runs per cell; Wilson intervals in `reports/exp-09-control-competency.md`. Bold marks scenarios where uniform random beats every search strength. No agent exceeds 0.39 anywhere, and the $N=256$ vs. $N=16$ difference is at most 0.02 on every scenario.

Three pre-registered structural claims all held. No scenario exceeds 0.50 correct at any N (maximum observed: 0.39). The rates are *flat in N* : sixteen times more simulation buys no tactical competence. And uniform random outright beats every search strength on three of the five scenarios — because random sometimes does nothing by accident, while search systematically prefers the visible action. The mechanism is directly observable in the decision logs. On S2, search cast the board wipe at the first legal moment, wiping exactly the two visible creatures, in **300 of 300 runs at every strength**: the “cast now” branch shows two guaranteed kills in every determinized world, and the “wait” branch’s better wipe never materializes in random continuations. This is strategy fusion [Frank and Basin, 1998] in its purest observable form. On S3, the Bolt was spent in 100% of runs before the key threat was even castable, and the threat died in zero of 400 runs across all agents. On S1, search always spent the Counterspell and put it on the bait creature 61–70% of the time — the rollout pathology that prices a held counter at zero.

Win rate masks incapacity. The companion micro-format experiment is the sharpest measurement lesson the platform has produced. A minimal control deck against a minimal aggro deck, 300 seat-balanced games per cell: the control deck wins 3.7% under random piloting, 39.3% at SEARCH-16, 63.0% at SEARCH-64. Search *rescues the win rate* — while the per-decision probes show it never plays control. The statistics that improve with N are exactly the immediate-value ones (bolt-target quality among simultaneously visible creatures: $0.34 \rightarrow 0.69$); the statistics that require valuing the future are flat or *worse than random* at every N (holding instants: ≈ 0.40 vs. random’s 0.57; a quarter of all Counterspells still burned on one-mana creatures at $N=256$). Search- N buys discrimination *within* the present decision and none *across* turns. A rising win rate is not evidence that a policy has acquired the behavior a deck is built around.

Two consequences follow. First, the 22% matchup number says nothing about the decks: the pilot it was measured under cannot execute any of the five competencies the control deck is built on, so the deck hypothesis is unmeasurable with this pilot. Second, the suite is the **acceptance test for Challenge 2**: a proposed fix to imperfect-information search — learned beliefs, information-set planning, policy-guided rollouts — must at minimum lift S2 and S5 off zero, and should report all five scores. Any agent scoring near the search rows should not be called a control player, whatever its ladder position.

7 The Challenge Areas

These five problems organize the gaps exposed by the current platform. They are not an exhaustive agenda for *Magic AI*, and solving them on this subset would not imply strong full-game play. Each challenge has a current result, a measurable entry criterion, and a concrete first experiment.

Challenge 1: Beat the search ladder

Statement. Produce a policy that, at inference time and without search, has ladder strength $N \geq 16$.

Current best. `student_r0` (§5.1), estimated ladder strength $N \approx 7$ for $\approx \$2.90$ in the current world; `bc-search64` reached $N \approx 8$ for \$0.66 in the pre-Milestone-1 world. Both rungs are descriptive rather than confirmed (each interval at the nearest rung includes 50%). The terminal-only and potential-shaped PPO policies remain unmeasured on the ladder.

Entry criterion. Beat SEARCH-16 head-to-head over ≥ 400 seat-balanced games with a Wilson lower bound above 50%, at < 10 ms per decision and $< \$5$ of training. Report ladder strength, dollars, and milliseconds.

Why it is tractable, and what we know does not work. One round of argmax behavior cloning — no DAgger [Ross et al., 2011], value head, soft targets, or iteration — produces a learned policy near a measured ladder rung, cheaply and reproducibly. The obvious escalation, expert iteration [Anthony et al., 2017, Silver et al., 2018], has now been tried once and *degraded* the student (§5.1); at this scale the loop’s binding constraint is the cost of good labels, not the algorithm’s shape. Routes that remain open: a bigger random-rollout teacher budget, value-guided search that replaces rollouts with one forward pass at the leaves, and cheap fast-rollout policies.

Challenge 2: Model the belief state

Statement. Replace uniform determinization with a learned posterior over the opponent’s hidden state, and demonstrate that it improves play at a matched simulation budget.

Current best. No learned belief model in manabot. `determinize()` draws uniformly from the opponent’s unseen pool. This is correct before public behavior conveys information and increasingly misspecified when actions depend on private cards.

Entry criterion. A belief model $b(\text{hand} \mid \text{public history})$ that (i) beats the uniform prior in held-out log-likelihood of the true opponent hand, (ii) when substituted into `determinize`, yields a searcher that beats an equal-simulation uniform searcher over ≥ 400 seat-balanced games, and (iii) reports the control-competency suite (§6), lifting hold-the-wipe and hold-up-quench strictly off zero.

Why it is tractable, and what it would not solve. Public decklists provide a known card multiset, and the simulator exposes the true hidden state at every self-play history, so supervised targets are available without hand annotation. The engine also isolates sampling inside `determinize()`, making a posterior substitution local. A learned posterior would address *which worlds are sampled*; it would not by itself fix strategy fusion [Frank and Basin, 1998], where perfect-information plans differ across states the player cannot distinguish. Both failure modes are now directly observed in this platform (§6): the scenario suite caught strategy fusion executing the premature board wipe in 300 of 300 runs, and the rollout pathology burning held counters on bait. Information-set planning methods such as ISMCTS [Cowling et al., 2012a] and ReBeL [Brown et al., 2020] address the fusion half; the suite is the acceptance test for any of them.

Challenge 3: Long-horizon credit assignment from terminal reward

Statement. Train an agent from terminal reward alone to ladder strength $N \geq 16$.

Current best. Terminal-only PPO reaches 75.5% against random (best of three seeds, \$0.14) but has not been placed on the ladder.

Entry criterion. As stated, using only the terminal game outcome as reward. Shaped rewards address a different question and are outside this challenge.

Why it is hard. On the interactive random-play profile, one player sees about 58 surfaced decisions, versus a GAE window of ≈ 16.8 steps. Per-event shaping performs worse than terminal-only reward because it rewards a cast-and-race strategy that does not fit the deck (§9). Potential-based shaping [Ng et al., 1999] removes that prior but does not establish a finite-budget improvement over terminal-only PPO in three seeds. Longer-range value learning, recurrence, hierarchy, and search-derived targets remain untested routes.

Challenge 4: Compositional generalization to unseen cards

Statement. Train an agent that plays competently with cards it has never seen.

Current best. No zero-shot game-play result in manabot. The observation encoder represents cards as hand-built feature vectors: type flags, keyword flags, ward and kicker costs, power/toughness/mana value, and type tags. Related work has demonstrated generalized representations for unseen cards in deck-construction decisions [Bertram et al., 2024], but transfer to sequential play remains open here.

Entry criterion. Hold out the 26-card *Avatar* slice (`cardsets/tla.rs`) from training, freeze the policy, and evaluate it on decks containing those cards. It must beat an otherwise matched identifier-only or randomized-representation baseline head-to-head with a Wilson lower bound above 50%; also report the gap to an oracle trained with the held-out cards.

Why it is approachable. *Magic* cards pair natural-language rules text with engine-level structure, and the policy already scores actions through the objects they reference. That isolates the card encoder as a replaceable component. A text or program encoder does not provide compositional generalization for free; the held-out-card protocol is intended to distinguish genuine transfer from a larger lookup table.

Challenge 5: Scale the rules substrate

Statement. Grow the engine from 55 cards toward a playable cube while preserving rules-test coverage and throughput.

Current best. 55 cards, 162 rules-cited tests, 1.8×10^5 steps/s (measured on the original pool). Milestone 1 (§3.5) cleared the first hard slice — mid-resolution choices, an additive layers subset, earthbend, exile linkage — with the suite green. Full CR 613 (timestamps, dependency, non-additive effects), replacement and prevention effects (CR 614), mulligans, player-selected combat-damage ordering, and floating-mana choice remain absent.

Entry criterion. Implement the next CR slice (replacement/prevention being the hard one now), keep the rules suite and the Scryfall conformance tripwire green, and hold $\geq 10^5$ steps/s at 16 environments.

Why it matters. Rules coverage gates every other challenge. Our trigger substrate (§3.4) is a bet that triggered abilities decompose into subjects, predicates, and composable effects; that bet survived its first 26-card expansion intact — the defects the expansion surfaced were in hand-transcribed card data, not in the substrate (§9). Whether it survives non-additive continuous effects is the next deliberately hard test.

8 Scope and Limitations

Game scope. Managym implements a 55-card, two-player subset without mulligans, replacement or prevention effects, full layers, or several combat and mana choices. The experiments use two fixed 60-card mirrors plus one 40-card constructed matchup and two micro-format decks; all frontier results in Table 2 use INTERACTIVE, while one calibration replication also uses STANDARD. Results should not be read as estimates of performance on unrestricted *Magic*.

Opponent scope. Absolute win rates are anchored on uniform random play. The ladder uses our own flat determinized search, whose rollouts also assume uniform random play and inherit perfect-information Monte Carlo pathologies — pathologies that §6 now measures directly: the ladder’s own reference agents fail all five competency scenarios. There is no adaptive self-play population, Elo pool, external agent, or human evaluation. The current ladder is useful for regression and within-project comparison, not evidence of competitive strength, and ladder position and behavioral competence are demonstrably separable axes.

Comparison scope. Several comparisons are not fully controlled. The distilled policy has twice the parameter count of the cost-matched PPO policy; the matched PPO uses a reward recipe later shown to be harmful; policy sampling during evaluation is stochastic; and the strongest PPO policies have not been placed on the ladder. The tables preserve these results as a baseline ledger, but causal claims about learning algorithms require controlled follow-up experiments.

Research scope. The paper studies game play with fixed decks. Deck construction and drafting are separate problems with their own prior work [Ward and Cowling, 2009, Bertram et al., 2024]; neither is implemented here. These limitations are not peripheral: broad card coverage, stronger opponents, and deck construction determine whether progress on this subset transfers to the game people play.

9 Calibration Findings, or: What This Domain Will Do To You

Table 4 lists fourteen claims that influenced the project and the evidence that overturned them. Several originated in unbalanced, single-seed measurements; others were assumptions that profiling, replication, or an oracle had never tested. They are included because they changed architecture, reward, or evaluation decisions, and because each was cheaper to falsify than to build around.

Claim we acted on	Killed by	What was actually true
“The agent wins 82% against random.”	seat balance	It was on the play. Random wins 94% on the play.
“Terminal reward fails; it causes pass-collapse.”	replication	Does not replicate on either deck. Terminal-only is among our best recipes.
“Per-event shaping is needed because the GAE horizon is too short.”	reward ablation	Per-event shaping is harmful; potential-based and terminal-only are not separated here.
“ <code>cast_when_able</code> ≥ 0.70 marks a healthy policy.”	INTERACTIVE	The interpretation changes with the deck; strong interactive baselines can cast less.
“Initialization variance is catastrophic (13–89%).”	seat balance	A <code>seat</code> $\times$ <code>init</code> interaction. Balanced: 48.2–51.7%; all CIs cover 50%.
“Most decisions are trivial and auto-passed.”	profiling	Only 12.4% collapse. Combat is 58% of real decisions.
“The environment is the bottleneck.”	profiling	In the profiled stack, inference costs $72\times$ the environment step.
“The interactive deck surfaces more decisions.”	profiling	They fall to $0.60\times$.
“Search narrows a cross-deck gap by reclaiming conditional value.”	matchup table	Search <i>widened</i> it (UR 34.5% $\rightarrow$ $\approx 23\%$); random rollouts price held interaction at zero.
“The 22% matchup number measures the decks.”	competency suite	The pilot cannot execute a single control line; the number measured the pilot.
“Win rate tracks behavior.”	behavior probes	Micro-format win rate rose $0.04 \rightarrow 0.63$ with N while every cross-turn behavior stayed pathological.
“Hand-transcribed cards are close enough.”	oracle audit	20 of 55 card shells were wrong, including a wrong-color card and a card name that does not exist.
“One crank of expert iteration climbs the ladder.”	head-to-head	Round 1 lost 74–26 to round 0; cheap policy-rollout labels are worse than many random playouts.
“The GPU speeds up inference.”	profiling	Only at batch ≥ 256 ; and MPS does not multiplex across processes.

Table 4: Fourteen claims and their disposition. Each changed after a measurement, replication, or audit step, before another algorithmic intervention.

Several of these deserve their own statement, because they are findings about the domain rather than about us.

Per-event shaping installs the wrong prior. Three reward regimes, three seeds each, 400 seat-balanced games per seed:

Reward	Mean win %	Range	cast_when_able
Per-event shaping (our founding recipe)	52.4	43.2–58.3	0.88–0.95
Potential-based Φ	69.5	65.2–75.0	0.35–0.38
Terminal-only	66.7	60.0–75.5	0.15–0.29

Seat-balanced evaluation against uniform random on INTERACTIVE. Three seeds do not separate potential- Φ from terminal-only.

Every potential-shaped seed beats every per-event seed, with non-overlapping intervals. Terminal-only and potential- Φ overlap at this sample size. The policy-invariance result of Ng et al. [1999] concerns optimal policies under its assumptions; it does not imply that potential shaping has no effect on finite-budget learning. These experiments therefore support a narrower operational choice: terminal-only reward is the simpler default until a powered comparison shows that potential shaping improves sample efficiency. The pass-collapse observation that originally motivated dense shaping does not replicate on either deck.

The deck is the probe. The most informative number above is the last column. The *same* terminal-only reward yields `cast_when_able` ≈ 0.99 on STANDARD, where immediate development is usually useful, and 0.15–0.29 on INTERACTIVE, where holding interaction can be valuable. By contrast, two of three per-event seeds cast at 0.88–0.95 on the interactive deck. The paired profile distinguishes deck-sensitive behavior from a reward-induced habit; the competency scenarios of §6 are the direct tactical metrics this observation called for.

The pilot confounds every matchup table, and so does raw card quality. The first cross-deck table (§3.5) looked like a deck-strength measurement — until the competency suite showed the measuring instrument could not execute the losing deck’s game plan, making the deck hypothesis unmeasurable with that pilot. The complementary confound is card quality: in a follow-up probe on the same matchup, swapping a single card name — four copies of *Ancestral Recall* — into the weaker deck moved the SEARCH-64 mirror by 55 points, from 23.7% to 78.25%. At this pool size a matchup table is a joint measurement of decks, pilot, and a handful of over-powered cards, and attributing its numbers to any one factor requires the other two to be probed separately.

Game content needs CI against its source of truth. A full audit of every registered card against Scryfall oracle data found **20 of 55 card shells wrong**: a card implemented in the wrong *color* with the wrong cost and stats (*Glider Kids*, registered as a blue $\{1\}\{U\}$ 2/1 without flying; the real card is a white $\{2\}\{W\}$ 2/3 flyer — it had been sitting in blue-red test decks), a card whose name does not exist (“Grey Ogre”; the Alpha card is *Gray Ogre*), wrong power/toughness on a ward proof card, and over a dozen stale or empty text boxes. None of these were rules-engine bugs; all were hand-transcription drift that the rules-test suite could not catch, because the tests exercised the transcribed data. The fix is structural, not heroic: a committed Scryfall snapshot and a CI test that fails any registration whose shell diverges from it — and fails any *new* registration that lacks a fixture entry. Hand-transcribed game content is data, and data needs the same tripwires as code.

10 Getting Started

The repository contains the engine, the learner, the search player, the distillation harness, the evaluation protocol, and every experiment report backing the numbers in this paper.

- `managym/` — the Rust engine, its 162 rules-cited tests, and the Scryfall conformance tripwire. `cargo test`.
- `manabot/` — Gymnasium wrapper, PPO, `FlatMCPlayer`, the batched inference driver, distillation.
- `manabot/verify/` — the seat-balanced evaluation harness and the control-competency suite (`competency.py`). Use these; do not write your own.
- `reports/` — one report per experimental cycle; checked-in raw JSON is included for the search, distillation, shaping, expert-iteration, matchup, and competency evaluations.
- `docs/rules_coverage.yaml` — machine-tracked scope. If your card needs a rule not listed here, it is Challenge 5.

The fastest path to a comparable result is to reproduce SEARCH-16 against random, then place a policy on the ladder with the same seat-balanced protocol. A confirmed result beyond $N = 16$ clears Challenge 1; the competency suite is a four-minute run and tells you whether your agent can do what its win rate implies.

11 Conclusion

We set out to train an agent to play *Magic: The Gathering*, and built an apparatus for measuring progress instead. The engine is fast, deterministic, and tested within a stated subset. Determinization and state injection are native, so a flat imperfect-information search is one call and a scored tactical scenario is a few more. Seat balancing, replication, and an oracle audit overturned early conclusions in reward design, evaluation, and the card data itself.

On the interactive mirror, the cheapest search reaches 91.7% against random, while the best learned-from-scratch point estimate is 75.5%; the strongest learned policy overall is distilled from search (87.0%, ladder $N \approx 7$), and the one attempt to iterate that recipe degraded it. Meanwhile the search family that anchors the ladder fails all five control-competency scenarios — uniform random beats it on three — so the platform now measures a second axis, behavior, on which no current agent scores at all. This is a local baseline, not a verdict on learning. It defines the next experiments: stronger opponents, history-conditioned beliefs, information-set planning judged by the competency suite, zero-shot card transfer, and a broader rules substrate. The instrument is usable; the measurements reveal how much remains.

Reproducibility. All experimental cycles have reports under `reports/`; checked-in JSON under `reports/data/` covers the search, distillation, shaping, expert-iteration, two-deck-matchup, and competency evaluations, while earlier reports give provenance and reproduction commands. Provenance commits: `e38f20b` (cost basis), `b884038` (decision profile), `f019e7a` (seat-balanced baselines). Win-rate intervals are Wilson 95%.

References

- Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Timo Bertram, Johannes Fürnkranz, and Martin Müller. Learning with generalised card representations for Magic: The Gathering. In *IEEE Conference on Games (CoG)*, 2024.

- Noam Brown, Anton Bakhtin, Adam Lerer, and Qucheng Gong. Combining deep reinforcement learning and search for imperfect-information games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- Alex Churchill, Stella Biderman, and Austin Herrick. Magic: The Gathering is turing complete. In *10th International Conference on Fun with Algorithms (FUN)*, 2021.
- Peter I. Cowling, Edward J. Powley, and Daniel Whitehouse. Information set monte carlo tree search. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(2):120–143, 2012a.
- Peter I. Cowling, Colin D. Ward, and Edward J. Powley. Ensemble determinization in monte carlo tree search for the imperfect information card game Magic: The Gathering. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(4):241–257, 2012b.
- Cristiano da Costa Cunha, Wei Liu, Tim French, and Ajmal Mian. Causal reinforcement learning for complex card games: A Magic: The Gathering benchmark. *arXiv preprint arXiv:2605.06066*, 2026.
- Ian Frank and David Basin. Search in games with incomplete information: A case study using bridge card play. *Artificial Intelligence*, 100(1–2):87–123, 1998.
- Shengyi Huang, Rousslan Fernand Julien Dossa, Chang Ye, Jeff Braga, Dipam Chakraborty, Kinal Mehta, and João G.M. Araújo. Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms. *Journal of Machine Learning Research*, 23(274):1–18, 2022.
- Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011.
- John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

- Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- Colin D. Ward and Peter I. Cowling. Monte carlo search applied to card selection in Magic: The Gathering. In *IEEE Symposium on Computational Intelligence and Games*, pages 9–16, 2009.
- Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.